Part 1 – Theoretical Questions

1. ReactJS and NextJS: Rendering

a) Differences

With CSR (Client-Side Rendering), the server sends almost empty HTML, and JavaScript runs in the browser to build the page. The content is displayed after the JavaScript has loaded and finished executing.

With SSR (Server-Side Rendering), the server generates the HTML on every request and sends the fully rendered page to the browser. Then, React hydrates the HTML in the browser to make it interactive.

With SSG (Static Site Generation), the HTML is generated only once, during the build process, and every user receives the same static file. It can be served through a CDN, making it very fast.

b) When to use each approach

CSR works well for authenticated areas, dashboards, and internal applications where there is a lot of interaction and SEO is not important. The main disadvantages are the initial blank screen and poor SEO performance for public pages.

SSR is suitable for pages that depend on frequently changing data or user-specific information and also need to be indexed by search engines. A good example is a product page with updated prices and stock availability. The downside is that the server processes every request, which can make it slower and more expensive.

SSG is ideal for blogs, documentation, and landing pages where the content does not change very often. The main disadvantage is that the content can become outdated until the next build. It is also not suitable for pages that depend on the currently logged-in user. Additionally, generating thousands of pages can become expensive. Next.js provides ISR (Incremental Static Regeneration), which allows pages to be regenerated periodically and helps reduce this problem.

In this challenge, I used **SSR for the task list** because the list changes whenever a task is created or deleted, and I wanted the HTML to already contain the tasks when it reaches the browser.

2. Backend

a) Different ways to implement a backend

There are several ways to implement a backend:

- A separate REST API using frameworks such as Express, Fastify, or NestJS.
- GraphQL, when the frontend needs to select exactly which fields it wants to receive.
- Next.js built-in features, such as Route Handlers and Server Actions, allowing the frontend and backend to stay in the same project.
- tRPC, which also runs inside the project and exposes backend functions as type-safe procedure calls.
- Backend-as-a-Service solutions such as Firebase or Supabase, when I don't want to build and maintain the server myself.
- Serverless functions, which execute only when they are invoked.

b) tRPC

I am familiar with tRPC, and I used it in this challenge.

Its main advantage is **end-to-end type safety**. If I change a return type in the backend, the frontend can detect the mismatch immediately, without requiring me to manually write API contracts or generate code.

Together with Zod, input data can also be validated. This helped me ensure that users cannot create tasks without a title.

Regarding security, tRPC does not provide security automatically. Types only exist at compile time, so server-side validation with Zod is essential. Authentication must also be implemented separately using context and middleware, because procedures are public by default unless access control is configured. Rate limiting must also be handled separately.

Regarding scalability, tRPC is only the communication layer, so it scales along with the server where it runs. The main risk is related to code organization: if everything is placed in a single router, the codebase can quickly become difficult to maintain. The ideal approach is to separate routers by domain and keep business logic inside services, which is the structure I used in this challenge.

One disadvantage is that the frontend and backend become tightly coupled and both need to use TypeScript, preferably within the same project. If I need to expose the API to a mobile application written in another language or to external clients, REST or GraphQL would be more suitable alternatives.

Another disadvantage is that tRPC does not take advantage of standard HTTP caching as easily as a traditional REST API.

Part 2 – Practical Challenge

````
## Instalação

Requisitos: Node.js e npm.

```bash
git clone https://github.com/neemias-araujo-dev/task-api.git
cd task-api
npm install
npm run dev
```

Abra [http://localhost:3000](http://localhost:3000) no navegador.

Para gerar e iniciar a versão de produção:

```bash
npm run build
npm run start
```
````